

Parallax: Community-Structured Graph Attention for Knowledge Graph Reasoning

A White Paper

Authors: Bryan (Originator), Claude Sonnet 4.6 (Research Collaborator)

Date: March 2026

Status: Pre-publication draft ? concept and architecture stage

License: Proprietary ? all rights reserved

Abstract

We propose Parallax, a novel framework that enables Knowledge Graphs (KGs)

to perform multi-hop reasoning using the same structural principles that make

Transformer-based Large Language Models powerful ? without requiring an LLM,

without training data, and with full interpretability of every inference step.

The central contribution is Community-Structured Attention (CSA): a

mechanism in which graph communities serve as attention heads, graph traversal

replaces matrix multiplication, and hop depth replaces layer depth. Unlike

Graph Attention Networks (GATs), which apply learned attention within hard

adjacency constraints, CSA uses community membership as a soft global

constraint that captures both local topological cohesion and global structural

significance simultaneously.

This is made possible by a second contribution: the **Dual-Signal Community**

Fusion (DSCF) algorithm, which produces communities that encode both LPA

majority-vote structure (local) and modularity gain (global) in a single

partition. We show that DSCF communities possess a structural duality that

maps naturally to the dual character of multi-head attention in Transformers.

Together, CSA and DSCF form an architecture where a KG can answer multi-hop

questions by traversing itself, with every reasoning step grounded in explicit

graph edges, every conclusion traceable to a path, and no LLM required for

inference ? though one may optionally be used for natural language generation.

1. Introduction

1.1 The Gap Between Knowledge Graphs and Language Models

Knowledge Graphs and Large Language Models represent two fundamentally different approaches to knowledge representation and reasoning.

Knowledge Graphs store knowledge explicitly: entities as nodes, relationships

as typed edges, facts as (subject, predicate, object) triples. This makes

them precise, verifiable, and updatable without retraining. However, they

cannot reason beyond what is explicitly stored. Multi-hop inference ? "Marie

Curie discovered Polonium, Polonium is radioactive, therefore Marie Curie

discovered a radioactive element" ? requires either hardcoded graph traversal

queries or external reasoning systems.

Large Language Models store knowledge implicitly in billions of weight

parameters. They can reason, generalize, and synthesize across domains. But

this implicit representation is opaque, cannot be updated without expensive

fine-tuning, and is prone to hallucination ? generating plausible-sounding

but incorrect facts with no mechanism for ground-truth verification.

The field has responded with hybrid approaches: Retrieval-Augmented Generation

(RAG), Knowledge-Graph-augmented LLMs, and GraphRAG. In all of these, the KG

is a retrieval store and the LLM does the reasoning. The KG remains passive.

Parallax inverts this relationship. The KG reasons. The LLM, if present,

only generates natural language from the KG's output. Every inference step is

a graph traversal, every conclusion is a path, and every path can be verified.

1.2 The Core Observation

A Transformer's power comes from three mechanisms working together:

Multi-head attention: different heads specialize on different

relational aspects of the input (syntactic, semantic, long-range, etc.)

Deep composition: each layer builds on the previous, allowing complex

multi-step reasoning

Positional awareness: the model knows where each token sits relative

to others

We observe that Knowledge Graphs have natural analogs for all three:

Community structure serves the role of attention heads: nodes within

a community have strong mutual relevance; communities specialize on

different conceptual domains.

BFS hop depth serves the role of layer depth: each hop is one step of

composed reasoning.

Graph-structural features (PageRank, betweenness, degree) serve the

role of positional encoding: they tell the model where each entity sits in

the global information landscape.

The question is: can these analogs be made operational ? not merely

metaphorical? We argue yes, and demonstrate the architecture to do so.

1.3 Contributions

This paper makes three primary contributions:

The Parallax architecture: a complete mapping of Transformer components

to KG operations, enabling multi-hop reasoning via graph traversal alone.

Community-Structured Attention (CSA): a novel attention mechanism using

community membership as a soft global constraint on graph traversal,

bridging the gap between local GAT-style attention and global Transformer-

style attention.

Dual-Signal Community Fusion (DSCF): a novel community detection

algorithm combining LPA majority-vote and modularity gain simultaneously

at each node update, producing communities with dual short-range/long-range

character that maps to multi-head attention's dual specialization.

2. Background and Related Work

2.1 Graph Neural Networks

Graph Neural Networks (GNNs) [Scarselli et al., 2009] generalize neural

networks to graph-structured data. The message-passing paradigm [Gilmer et

al., 2017] defines node updates as aggregations of neighbor representations.

Graph Attention Networks (GATs) [Velickovic et al., 2018] introduce learned

attention weights between connected nodes. Key limitations for our purposes:

(a) attention is restricted to direct neighbors ? no global context;

(b) communities are not considered; (c) training labels required.

GraphSAGE [Hamilton et al., 2017] samples and aggregates local neighborhoods

but similarly lacks global structural awareness.

None of these use community structure as an organizational principle for attention.

2.2 Knowledge Graph Reasoning

Early KG reasoning systems used rule-based approaches (AMIE [Garraga et al., 2013]) or embedding-based methods (TransE [Bordes et al., 2013], RotatE [Sun et al., 2019]). These methods produce entity embeddings but do not perform multi-hop traversal in the attention-mechanism sense.

Path-based reasoning approaches (DeepPath [Xiong et al., 2017], MINERVA [Das et al., 2018]) use reinforcement learning to find paths. These are closer to our goal but require training data and do not use community structure.

2.3 LLM + KG Hybrid Systems

KG-GPT [Yao et al., 2023], KGPT [Chen et al., 2020], and related systems connect KGs to LLMs as retrieval stores. The LLM always performs the

reasoning step; the KG is passive.

GraphRAG [Edge et al., 2024] uses community detection (Leiden)

to summarize graph clusters as text, which is then passed to an LLM for

RAG. This is the closest existing work. However:

Communities are summarized as static text chunks, not used as attention heads

The LLM performs all reasoning

Paths are not returned or made interpretable

The system is not graph-agnostic (designed for Microsoft's pipeline)

RAPTOR [Sarathi et al., 2024] builds hierarchical text clusters for RAG but

operates on documents, not graphs, and uses tree structure rather than

community structure.

2.4 Community Detection

The Louvain algorithm [Blondel et al., 2008] optimizes modularity greedily

but can produce internally disconnected communities. Leiden [Traag et al., 2019] fixes this with a refinement phase guaranteeing internal connectivity.

Label Propagation [Raghavan et al., 2007] is fast and unsupervised but non-deterministic and produces variable quality.

DSCF (this work) combines LPA and Leiden signals simultaneously at each node update, using a temperature-annealing schedule. See Section 4.2.

DSCF is non-deterministic (inherits LPA's shuffle-order sensitivity).

For reproducible results: run 3-5 trials and select the partition with highest modularity score. For production: seed the RNG and document the seed.

3. The Structural Equivalence

We establish a complete operational mapping between Transformer components and KG operations. This is not analogy ? each mapping is functional.

Transformer Component

Parallax (KG) Equivalent

Notes

Token

Entity or Relation

Atomic unit of information

Vocabulary

Entity type taxonomy

Closed set of possible types

Token embedding

Entity embedding (TransE/RotatE)

Dense vector per entity

Positional encoding

Structural encoding (PR, BW, deg)

Where entity sits globally

Attention head

Community cluster (DSCF)

Specialized relational context

Attention weight

CSA weight formula

Sim + community + edge + distance

Context window

Ego-network radius R

How far to traverse

Layer depth L

BFS hop count

Reasoning step count

Feed-forward sublayer

Entity-type projection

Type-specific transformation

Residual connection

Previous-hop embedding

Prevents information loss

Layer normalization

Embedding normalization

Prevents value explosion

Output projection

Path decoder / ranker

Maps traversal to answer

KV cache

Materialized path store

Reuse traversal across queries

This equivalence has a critical implication: **the number of attention heads is**

not a hyperparameter in Parallax ? it is determined by the graph's own community

structure. **A graph with 12 natural communities has 12 attention heads. A graph**

with 200 communities has 200. The architecture adapts to the data.

4. The Parallax Architecture

4.1 Community-Structured Attention (CSA)

CSA computes attention weights for graph traversal that incorporate both local

graph topology and global community structure.

Attention weight formula:

For entity u attending to entity v at traversal hop k :

$a(u, v, k) = ?$

$$\begin{aligned}
 & \alpha \cdot \text{cosine_sim}(\text{emb}(u), \text{emb}(v)) \\
 & + \beta \cdot \text{community_score}(u, v) \\
 & + \gamma \cdot \text{edge_type_weight}(\text{type}(u, v)) \\
 & - \delta \cdot \text{normalized_distance}(u, v) \\
 & + \epsilon \cdot \text{hop_decay}(k) \\
 &)
 \end{aligned}$$

Where:

$\text{emb}(\cdot)$ is the entity embedding (any KGE method or sentence encoder)

$\text{community_score}(u, v)$:

1.0 if $\text{community}(u) == \text{community}(v)$ [same head]

0.5 if communities are adjacent [neighboring heads]

$\exp(-\delta \cdot \text{community_distance}(u, v))$ otherwise [distance decay]

$\text{edge_type_weight}(\text{type})$: learned or manually assigned per relation type

$\text{normalized_distance}(u, v)$: shortest path length / graph diameter

$\text{hop_decay}(k)$: encourages shorter paths (e.g., $1 / (1 + k)$)

σ : sigmoid activation

$\alpha, \beta, \gamma, \delta, \epsilon$: tunable parameters

Default parameter values (zero-shot deployment):

$\alpha = 0.4$ (embedding similarity)

$\beta = 0.4$ (community membership)

$\gamma = 0.1$ (edge type)

$\delta = 0.05$ (distance penalty)

$\epsilon = 0.05$ (hop decay)

Parameters can be learned from (query, answer) pairs for supervised settings.

community_score definition (complete):

$\text{community_score}(u, v)$:


```

if community(u) == community(v):    return 1.0

if communities_are_adjacent(u, v):    return 0.5

else:

    d = community_distance(u, v)

    return exp(-? · d)

community_distance(u, v):

    # Shortest path (in hops) between the community of u and community of v

    # in the community-level graph, where communities are nodes and two

    # communities are adjacent if ?1 cross-community edge exists between them.

    # Precomputed once after DSCF converges via BFS on the community graph.

    # ? = 0.5 default (controls cross-community decay rate)

```

Why this is not a GAT:

GATs compute $a(u, v) = f(W_u \cdot \text{emb}(u), W_v \cdot \text{emb}(v))$? purely from learned

weights on adjacent node pairs. They cannot express the community membership

term $\gamma \cdot \text{community_score}(u, v)$, which introduces global structural awareness

without requiring the full $O(n^2)$ attention of Transformers.

CSA is $O(n \cdot k \cdot C)$ where k is average degree and C is the average number

of community-adjacent entities to consider γ far cheaper than Transformer

attention while capturing global structure via community membership.

4.2 Dual-Signal Community Fusion (DSCF)

DSCF is the community detection algorithm that produces the attention head

structure. It is a key contribution in its own right.

The core innovation: at each individual node update, both the LPA

majority-vote signal (local topology) and the modularity gain signal (global

structure) are computed. The decision incorporates both simultaneously,

governed by a temperature parameter:

For each node v at each iteration:

1. LPA signal:

$lpa_cid = \text{argmax over neighbor labels (majority vote)}$

$lpa_conf = \text{vote_count}[lpa_cid] / \text{total_neighbors} \in [0, 1]$

2. Modularity signal:

For each candidate community C adjacent to v :

$Q(v, C) = k_{\{v, C\}} / m \times \text{resolution} \times k_v \times k_C / (2m^2)$

$best_mod_cid = \text{argmax } Q$

$mod_conf = \min(best_Q \times m, 1.0) \in [0, 1]$

3. Decision:

if $lpa_cid == best_mod_cid$? current:

MOVE (consensus anchor ? high confidence)

elif both say STAY:

STAY

elif only LPA says MOVE:

MOVE with probability $\text{lpa_conf} \times \text{temperature}$

elif only modularity says MOVE:

MOVE with probability $\text{mod_conf} \times (1 + (1 - \text{temperature}))$

else (disagree on different targets):

$\text{lpa_weight} = \text{lpa_conf} \times \text{temperature}$

$\text{mod_weight} = \text{mod_conf} \times (2 - \text{temperature})$

MOVE to weighted-random choice

temperature schedule: $\tau_{t+1} = \max(\tau_t \times \text{cooling}, 0.01)$

Post-convergence: Leiden-style connectivity check & split any community whose

induced subgraph is disconnected.

Why DSCF communities are the right attention heads:

In a trained Transformer:

Some heads specialize on local structure (syntactic patterns, adjacent tokens)

Other heads specialize on long-range structure (coreference, semantic themes)

The combination allows both local and global reasoning simultaneously

DSCF communities exhibit exactly this dual character. The LPA component ensures

communities are locally coherent (nodes in the same community are topologically

close). The modularity component ensures communities are globally significant

(they represent structurally distinct regions of the graph). A node that is in

a DSCF community is there because both local and global signals agreed.

This dual property has not previously been used as a basis for attention

mechanisms in any published graph learning system.

Comparison to Leiden-only communities:

Leiden optimizes purely for modularity. On sparse regions of a graph, Leiden

may split locally coherent neighborhoods across multiple communities because

the global modularity gain favors a different partition. DSCF resists this ?

the LPA component holds locally coherent groups together even when modularity

would split them.

Comparison to LPA-only communities:

LPA can merge structurally distinct regions if they happen to be locally

connected (the "resolution limit" problem). DSCF resists this ? the

modularity signal penalizes over-merging.

5. The Forward Pass: Graph Reasoning

5.1 Algorithm

INPUT: Query Q (text string or entity list)

Graph G (any backend)

Community assignments C (from DSCF)

Entity embeddings E (from any KGE method)

Hop depth L , beam width B , top- K

OUTPUT: Ranked list of reasoning paths $P = [(path, score, explanation)]$

STEP 1 ? Entity Grounding

If Q is text: extract entities via NER or fuzzy match to graph vocabulary

$S = \{e_1, e_2, \dots, e_n\}$ // seed entities

$h_{e_i} = E[e_i]$ // initial embeddings

STEP 2 ? Structural Encoding

For each seed entity e_i :

$pos_i = [pagerank(e_i), betweenness(e_i), degree(e_i), community_id(e_i)]$

$h_{e_i} = LayerNorm(h_{e_i} + W_{pos} \cdot pos_i)$

STEP 3 ? Attention Traversal (L layers)

$beam = [(path=[e_i], embedding=h_{e_i}, score=1.0) \text{ for each } e_i \text{ in } S]$

For $k = 1$ to L :

candidates = []

For each (path, h, score) in beam:

current = path[-1]

neighbors = G.neighbors(current)

For each neighbor v in neighbors:

$w = \text{CSA}(\text{current}, v, k)$ // attention weight

$h_{\text{new}} = \text{ReLU}(W_k \cdot (w \cdot E[v] + h))$ // aggregation

// W_k : hop-depth projection matrix (dim $\times$ dim)

// Zero-shot default: $W_k = I$ (identity) ? no learned projection

// Supervised setting: W_k learned per-hop via path-ranking loss

$h_{\text{new}} += h$ // residual

$h_{\text{new}} = \text{LayerNorm}(h_{\text{new}})$

$\text{path_score} = \text{score} \times w \times \text{community_coherence}(\text{path} + [v])$

`candidates.append((path + [v], h_new, path_score))`

`beam = top_B(candidates, key=path_score)    // beam pruning`

STEP 4 ? Path Scoring

Final score for path $P = (e_1 ? r_1 ? e_2 ? r_2 ? \dots ? e_n)$:

$\text{score}(P) = ? \text{attention_weights}$

$\times \text{community_coherence}(P)$

$\times \text{semantic_alignment}(h_{\text{final}}, \text{query_embedding})$

STEP 5 ? Output

Return top-K paths ranked by score

Each path includes:

- Ordered entity/relation sequence
- Score breakdown (attention, community, semantic)

- Community sequence (which "heads" were traversed)
- Natural language explanation template

5.2 Community Coherence

The community_coherence term rewards paths that traverse communities in a

principled way:

$\text{community_coherence}(\text{path}) =$

$(\text{intra-community steps} \times 1.0 + \text{cross-community steps} \times 0.5)$

$/ \text{total steps}$

A path that stays within one community scores 1.0 (tight local reasoning).

A path that makes one community transition scores ~0.75 (one conceptual leap).

This prevents paths that jump incoherently across unrelated domains.

5.3 Interpretability

The output of Parallax is always a path through the KG. This is fundamentally

different from LLM reasoning in two ways:

Verifiable: every step is an explicit (entity, relation, entity) triple

that exists in the graph. The system cannot hallucinate a connection that isn't there.

Auditable: the community sequence tells you which conceptual domains

were traversed. A path that crosses from community "Clinical Trials" to

community "Drug Mechanisms" to community "Side Effects" is immediately understandable.

This property makes Parallax specifically valuable in high-stakes domains

(medical, legal, financial) where hallucination is unacceptable.

6. Implementation Architecture

6.1 Design Principles

Framework agnostic: Parallax must work with any graph database, any

embedding method, and any LLM (or no LLM). No vendor lock-in.

No training required by default: The zero-shot configuration uses fixed

parameters (?, ?, ?, ?, ?) and any entity embedding. Communities are

computed unsupervised via DSCF.

Progressive enhancement: Users can improve performance by providing

training pairs (for learning parameters) or domain-specific edge type weights

without changing the core architecture.

Minimal dependencies:

Core: networkx, numpy, leidenalg (for DSCF), scipy

Adapters: optional graph DB drivers

Embeddings: optional sentence-transformers or pykeen (for TransE/RotatE)

API: optional FastAPI

6.2 Repository Structure

parallax/

?

??? core/

? ??? __init__.py

? ??? graph_adapter.py # Abstract base class for graph backends

? ? ??? class GraphAdapter (ABC)

? ? ??? def get_neighbors(entity_id, edge_types=None) ? List[Edge]

? ? ??? def get_entity(entity_id) ? Entity

? ? ??? def get_edge_types() ? List[str]

? ? ??? def get_structural_features(entity_ids) ? Dict[str, Features]

? ?

? ??? embedding_engine.py # Entity embedding interface

? ? ??? class EmbeddingEngine (ABC)

```
? ? ??? class TransEEngine (EmbeddingEngine)
```

```
? ? ??? class SentenceEngine (EmbeddingEngine) # label-based, no training
```

```
? ? ??? class RandomEngine (EmbeddingEngine) # baseline/testing
```

```
? ?
```

```
? ??? community_engine.py # Community detection (DSCF + others)
```

```
? ? ??? def dscf_communities(G, resolution, max_iter, temp_start, cooling)
```

```
? ? ??? def leiden_communities(G, resolution)
```

```
? ? ??? def lpa_communities(G)
```

```
? ? ??? def hybrid_communities(G, resolution)
```

```
? ?
```

```
? ??? attention_engine.py # Community-Structured Attention
```

```
? ? ??? class CSAEngine
```

```
? ? ??? def compute_weight(u, v, k, communities, embeddings) ? float
```

```
? ? ??? def community_score(u, v, communities) ? float
```

```
? ?
```

```
? ??? structural_encoder.py # Graph positional encoding
```

```
? ??? def compute_pagerank(G) ? Dict[node, float]
```

```
? ??? def compute_betweenness(G) ? Dict[node, float]
```

```
? ??? def encode_features(features, dim) ? np.ndarray
```

```
?
```

```
??? reasoning/
```

```
? ??? traversal.py # Beam-search attention traversal
```

```
? ? ??? class BeamTraversal
```

```
? ? ??? def traverse(seeds, depth, beam_width) ? List[Path]
```

```
? ?
```

```
? ??? path_scorer.py # Multi-signal path ranking
```

```
? ? ??? def score_path(path, query_embedding, communities) ? float
```

```
? ? ??? def community_coherence(path, communities) ? float
```

```
? ?
```

```
? ??? answer_extractor.py # Extract top-K answers
```

```
? ??? def extract(paths, top_k) ? List[Answer]
```

```
?
```

```
??? adapters/
```

```
? ??? networkx_adapter.py # In-memory (default, no external deps)
```

```
? ??? neo4j_adapter.py # Neo4j via bolt
```

```
? ??? rdf_adapter.py # SPARQL endpoint (Wikidata, DBpedia)
```

```
? ??? csv_adapter.py # Bootstrap from edge-list CSV
```

```
?
```

```
??? llm_bridge/
```


? ??? context_formatter.py # Optional: format paths as LLM context

? ??? def to_prompt(paths, query) ? str

? ??? def to_structured(paths) ? dict

?

??? api/

? ??? server.py # FastAPI REST server

? ??? schemas.py # Pydantic models

?

??? cli/

? ??? parallax.py # Command-line interface

?

??? tests/

? ??? test_dscf.py

? ??? test_csa.py

? ??? test_traversal.py

? ??? fixtures/

? ??? toy_graph.csv # Small graph for unit tests

?

??? benchmarks/

? ??? webqsp_eval.py # WebQSP KG Q&A benchmark

? ??? metaqa_eval.py # MetaQA multi-hop benchmark

? ??? baseline_comparison.py # CSA vs GAT vs GraphRAG

?

??? examples/

? ??? wikidata_quickstart.py

? ??? neo4j_quickstart.py

? ??? csv_quickstart.py

?

??? pyproject.toml

??? README.md

??? PAPER.md # Living research document

6.3 The Abstract Graph Adapter

The adapter pattern is what makes Parallax truly agnostic:

```
from abc import ABC, abstractmethod
```

```
from dataclasses import dataclass
```

```
from typing import List, Optional
```

```
@dataclass
```

```
class Entity:
```

```
    id: str
```

label: str

type: str

properties: dict

@dataclass

class Edge:

source_id: str

target_id: str

relation_type: str

weight: float = 1.0

class GraphAdapter(ABC):

@abstractmethod

def get_entity(self, entity_id: str) -> Optional[Entity]: ...

@abstractmethod

```

def get_neighbors(self, entity_id: str,

    edge_types: List[str] = None,

    max_neighbors: int = 50) -> List[Edge]: ...

@abstractmethod

def find_entities(self, query: str, top_k: int = 10) -> List[Entity]: ...

@abstractmethod

def to_networkx(self) -> "nx.Graph": ... # for community detection

```

Any graph system that implements these four methods works with the full

Parallax stack. NetworkX, Neo4j, RDF SPARQL, property graph CSV ? all handled

by their respective adapter.

6.4 What Comes From AURA

The following components are directly portable from AURA with minor

generalization:

dscf_communities() ? pure Python, depends only on networkx

leiden_communities() / lpa_communities() / hybrid_communities()

Neo4j connection patterns and Cypher templates

The community broadcast WebSocket pattern (for live applications)

No other AURA-specific dependencies are carried over.

6.5 Phased Build Plan

Phase 0 ? Theory (complete)

Formalize CSA and DSCF; write white paper

Prototype DSCF in Python (done: lives in AURA knowledge_service)

Validate DSCF produces stable communities on AURA's Neo4j graph

Phase 1 ? Core Engine

core/graph_adapter.py ? abstract base + NetworkX adapter

core/community_engine.py ? DSCF, Leiden, LPA, hybrid (ported from AURA)

core/embedding_engine.py ? SentenceEngine (zero-training default)

core/attention_engine.py ? CSA weight formula

core/structural_encoder.py ? PageRank, betweenness, degree encoding

Unit tests on fixtures/toy_graph.csv for all of the above

Phase 2 ? Reasoning Engine

reasoning/traversal.py ? beam-search traversal with CSA weights

reasoning/path_scorer.py ? multi-signal scoring + community coherence

reasoning/answer_extractor.py ? top-K ranked answers

Integration test: end-to-end query on toy graph produces grounded paths

Phase 3 ? Adapters + API

adapters/neo4j_adapter.py (port AURA patterns)

adapters/rdf_adapter.py (SPARQL, for Wikidata/DBpedia)

adapters/csv_adapter.py (bootstrap from edge-list)

api/server.py ? FastAPI REST: /query, /communities, /health

cli/parallax.py ? command-line interface

Phase 4 ? LLM Bridge + Benchmarks

llm_bridge/context_formatter.py ? format paths as LLM prompts

benchmarks/webqsp_eval.py and metaqa_eval.py

Ablation study: DSCF vs Leiden vs LPA as attention heads

Baseline comparisons: BFS, GAT, GraphRAG, vanilla RAG

Phase 5 ? Publication

Write formal paper from white paper foundation

Submit to venue (e.g., EMNLP, ICLR, NeurIPS Graphs Track)

Open-source repository under chosen license

6.6 Computational Complexity

DSCF community detection: $O(E \cdot I)$ where E = edges, I = iterations

(typically $l < 50$ before convergence). Comparable to Louvain.

Community graph construction: $O(E)$ post-DSCF; precomputed once.

CSA weight per edge: $O(d)$ where d = embedding dimension. Precompute

structural encodings once; community lookups are $O(1)$ with a hash table.

Beam traversal (one query): $O(B \cdot L \cdot k \cdot d)$ where:

B = beam width (default 10)

L = hop depth (default 3)

k = average degree

d = embedding dimension

For a graph with $k=20$, $L=3$, $B=10$, $d=384$: $\sim 230,000$ floating-point operations

per query $\sim$ milliseconds on CPU, no GPU required.

Comparison to Transformer: Full attention is $O(n^2 \cdot d)$ per layer.

Parallax traversal is $O(B \cdot L \cdot k \cdot d)$ $\sim$ independent of graph size n .

This makes Parallax sublinear in graph size for fixed-width beam search.

6.7 Known Failure Modes

Dense hub nodes: Nodes with very high degree ($k \gg k?$) cause beam

explosion at that hop. Mitigation: cap `max_neighbors` per node (default 50).

Homogeneous graphs: If all nodes share the same community, CSA degenerates

to pure embedding similarity (`community_score` terms cancel). This occurs in

highly regular graphs (grids, complete bipartite). Mitigation: check community

count post-DSCF; if count < 3 , fall back to BFS with embedding similarity only.

Disconnected graphs: Multiple connected components each get their own

communities; cross-component traversal is impossible by definition. Queries

spanning components will return no paths. Mitigation: surface component

boundaries to callers; allow separate per-component queries.

Sparse embedding coverage: Entities with generic or missing labels get

near-random sentence embeddings. The γ term (embedding similarity) becomes noise; DSCF community structure (γ term) carries the full weight. Still functions; interpretability of similarity scores is reduced.

Adversarial community injection: A malicious actor who can insert edges into the graph can manipulate community structure and thus attention weights.

Relevant for applications where the KG is user-writable. Mitigation: sign trusted edges; treat unsigned-edge-induced communities with lower γ weight.

7. The DSCF-as-Attention-Head Hypothesis

The central theoretical claim of Parallax is that DSCF communities are better attention heads than Leiden-only or LPA-only communities. We state this as a falsifiable hypothesis:

H1 (DSCF Attention Hypothesis): For multi-hop reasoning tasks on KGs,

Parallax with DSCF attention heads achieves higher answer accuracy than

Parallax with Leiden-only or LPA-only attention heads.

H2 (CSA vs GAT Hypothesis): CSA-guided traversal achieves higher accuracy

on multi-hop questions than GAT-based traversal on the same graph and same

entity embeddings.

H3 (Interpretability Hypothesis): Parallax paths receive higher human

coherence ratings than equivalent LLM-generated reasoning chains on the same

questions, because every step is a grounded graph edge.

H3 Evaluation Protocol: Present matched pairs ? one Parallax path and

one LLM reasoning chain ? to N?30 annotators blind to their source. Ask:

"Which reasoning chain is more coherent and trustworthy? (A / B / Equal)".

Primary metric: proportion preferring Parallax path. Secondary: Cohen's kappa

for inter-annotator agreement (target ? > 0.6).

These hypotheses are testable on standard benchmarks (WebQSP, MetaQA-3hop)

and define the empirical work for Phase 2.

8. Open Research Questions

8.1 Embedding Strategy

Two options exist:

Option A ? Pre-trained structural embeddings (TransE/RotatE): trained

on the graph structure itself. More precise but requires a training step.

Suitable for static KGs or when training compute is available.

Option B ? On-the-fly label embeddings (sentence-transformers): encode

entity labels and descriptions using a pre-trained language model. No graph-

specific training needed. More agnostic. Less precise for entities with

ambiguous labels.

Recommended default: Option B for zero-shot deployment; Option A when the

graph has been stable and training is feasible. Parallax should support both

interchangeably via the EmbeddingEngine interface.

8.2 Adaptive Community Granularity

The DSCF resolution parameter controls how many communities are formed. Too

few communities = coarse attention heads that miss structure. Too many = noisy

heads that don't generalize.

Proposed adaptive rule: target $K \propto \sqrt{N}$ communities, where N = node count.

This is consistent with theoretical results on optimal modularity resolution

and ensures attention head count scales sensibly with graph size.

For AURA's KG ($N \approx 5,000$): target ~ 70 communities.

For Wikidata subset ($N \approx 100,000$): target ~ 316 communities.

8.3 Soft vs Hard Community Membership

DSCF produces hard assignments (each node belongs to exactly one community).

Real-world entities often span multiple communities $\rightarrow$ a person can be both

a scientist and a politician.

Extension: weight-based soft membership, where each node has a probability

distribution over communities. The `community_score` function becomes a

dot product of membership vectors. This would require modifying DSCF to track

confidence scores at convergence.

8.4 Learnable Parameters

In zero-shot mode, θ , ϕ , ψ , α , β are fixed. For supervised settings, they

can be learned from (query, ground-truth-answer) pairs via gradient descent

on a path-ranking loss. This is an optional enhancement that does not affect

the core architecture.

8.5 Temporal Knowledge Graphs

Time-stamped KGs (events, evolving relationships) introduce a temporal

dimension. The positional encoding would need to incorporate temporal distance

alongside graph-structural distance. Left for future work.

9. Benchmark and Evaluation Plan

9.1 Datasets

Dataset

Task

Hops

Size

WebQSP

Single + multi-hop QA

1-2

4,737 questions

MetaQA-2hop

Multi-hop QA

2

118,980 questions

MetaQA-3hop

Multi-hop QA

3

114,196 questions

FB15k-237

Link prediction

-

310,116 triples

Toy graph (internal)

Unit testing

1-4

~200 nodes

9.2 Baselines

System

Type

Notes

BFS (no attention)

Graph traversal

Traversal without CSA weighting

GAT

Graph neural network

2-layer, trained

GraphRAG

LLM-based

Community summaries ? GPT-4

RAG (vanilla)

LLM-based

FAISS retrieval ? GPT-4

Parallax (DSCF)

Graph attention

Ours, DSCF heads

Parallax (Leiden)

Graph attention

Ablation: Leiden-only heads

Parallax (LPA)

Graph attention

Ablation: LPA-only heads

9.3 Metrics

Hits@1, Hits@3, Hits@10: answer in top-K paths

Mean Reciprocal Rank (MRR): ranked answer quality

Path coherence (human eval): are the reasoning paths understandable?

Grounding rate: what fraction of returned paths are fully grounded

(all edges verified in the KG)?

10. Broader Impact and Applications

10.1 Domain Applications

Biomedical: Drug-gene-disease-pathway graphs. Multi-hop reasoning for drug

repurposing ("Drug X inhibits enzyme Y which is overexpressed in disease Z").

Grounded inference is critical ? no LLM should hallucinate drug interactions.

Legal: Case law citation and statutory reference networks. Multi-hop

precedent tracing. Every step in a legal argument must be citable; Parallax's

grounded paths match this requirement exactly.

Cybersecurity: Attack graphs, CVE dependency networks. "What path leads

from this exposed service to root access?" ? a life-safety question that

benefits from verified, traceable reasoning chains.

Software engineering: Code dependency and call graphs. Impact analysis:

"What does changing function X affect?" traversed as a multi-hop attention

path with community context (same module = high attention).

Finance: Entity relationship graphs for regulatory compliance. Traceable

reasoning chains for auditors: "Why did this transaction trigger a flag?"

10.2 The LLM Bridge

Parallax is designed to augment LLMs, not replace them. The `llm_bridge`

module formats traversal output as structured context:

You are reasoning about: [query]

The knowledge graph traversal found these paths:

Path 1 (score: 0.94):

Marie Curie [COMMUNITY: Scientific Discoveries]

? [discovered] ?

Polonium [COMMUNITY: Scientific Discoveries]

? [exhibits] ?

Radioactivity [COMMUNITY: Physics Phenomena]

Please summarize what this tells us about [query] in natural language.

This gives any LLM a grounded, structured context that minimizes the risk of

hallucination because the facts are provided explicitly. The LLM's role is

purely natural language generation, not reasoning.

10.3 The Agnosticism Property

Parallax is agnostic across five dimensions:

Graph database: implement GraphAdapter for any system

Embedding method: implement EmbeddingEngine for any model

LLM: any model or none ? Parallax works without one

Domain: the algorithm is domain-blind; community structure emerges

from the graph's own topology

Query language: entities can be identified from text, IDs, or direct

lookup ? the entry point is flexible

This means a single Parallax deployment can serve multiple graph backends

simultaneously, which is not possible with GraphRAG or KG-specific systems.

11. Conclusion

We have presented Parallax: a framework that enables Knowledge Graphs to

reason using the structural principles of Transformer attention without

training data, without an LLM, and with full interpretability.

The two core contributions ? Community-Structured Attention (CSA) and

Dual-Signal Community Fusion (DSCF) ? work together to give a KG the dual

character of multi-head attention: local cohesion (from DSCF's LPA component)

combined with global structural significance (from DSCF's modularity

component).

The resulting system produces reasoning paths, not black-box embeddings. Every

answer is traceable to a sequence of verified graph edges. Every reasoning step

names the community it traversed. This interpretability property, combined with

the zero-hallucination guarantee of graph-grounded inference, positions Parallax

as a meaningful complement to ? and in certain domains, replacement for ?

LLM-based reasoning over structured knowledge.

The open questions identified in Section 8 define the research program. The

benchmarks in Section 9 define the empirical standard. The architecture in

Section 6 defines what to build.

The name Parallax refers to the optical phenomenon where two viewpoints on

the same object yield depth perception that neither viewpoint alone provides.

LPA and modularity are two viewpoints on the same graph. Their combination

yields structural depth ? attention heads with both short-range and long-range

character ? that neither produces alone.

That depth is what makes the KG reason.

Appendix A: DSCF Algorithm ? Full Pseudocode

FUNCTION dscf_communities(G , resolution=1.0, max_iter=100,

temp_start=1.0, cooling=0.92):

$m = |E(G)|$

IF $m == 0$: RETURN $\{v\}$ for v in $V(G)$

nodes = list($V(G)$)

```
degree = {v: deg(v) for v in nodes}
```

```
// Singleton initialization
```

```
assignment = {v: i for i, v in enumerate(nodes)}
```

```
com_k = {i: degree[v] for i, v in enumerate(nodes)} // degree-sum cache
```

```
temperature = temp_start
```

```
FOR iteration = 1 to max_iter:
```

```
    changed = False
```

```
    SHUFFLE nodes
```

```
    FOR EACH v in nodes:
```

```
        neighbors = N(v)
```

```
        IF |neighbors| == 0: CONTINUE
```

```
        cur = assignment[v]; kv = degree[v]
```

```
        // LPA signal
```

```
vote = COUNT(assignment[nb] for nb in neighbors)
```

```
lpa_cid = argmax(vote); lpa_conf = vote[lpa_cid] / |neighbors|
```

```
// Modularity signal
```

```
candidates = {assignment[nb] for nb in neighbors} - {cur}
```

```
best_cid = cur; best_dq = 0
```

```
FOR cid IN candidates:
```

```
    k_vc = |{nb in neighbors : assignment[nb] == cid}|
```

```
    dq = k_vc/m - resolution × kv × com_k[cid] / (2m²)
```

```
    IF dq > best_dq: best_dq = dq; best_cid = cid
```

```
mod_conf = min(best_dq × m, 1.0)
```

```
// Decision
```

```
IF lpa_cid == best_cid ? cur:
```

```
    new = lpa_cid // consensus
```

```
ELIF best_cid == cur AND lpa_cid == cur:
```

```
    CONTINUE // both say stay
```

```
ELIF best_cid == cur:
```

```
    IF random() >= lpa_conf × temperature: CONTINUE
```

```
    new = lpa_cid
```

```
ELIF lpa_cid == cur:
```

```
    IF random() >= mod_conf × (1 + (1?temperature)): CONTINUE
```

```
    new = best_cid
```

```
ELSE:
```

```
    lpa_w = lpa_conf × temperature
```

```
    mod_w = mod_conf × (2 ? temperature)
```

```
    new = weighted_choice({lpa_cid: lpa_w, best_cid: mod_w})
```

```
// Apply move
```

```
com_k[cur] = max(com_k[cur] ? kv, 0)
```

```
com_k[new] = com_k[new] + kv
```

```
assignment[v] = new; changed = True
```

```
temperature = max(temperature × cooling, 0.01)
```

```
IF NOT changed: BREAK
```

```
// Connectivity post-pass (Leiden-style)
```

```
// For each community whose induced subgraph is disconnected:
```

```
// Split into connected components
```

```
// First component keeps the original community ID
```

```
// Additional components receive new IDs (max_existing_id + 1, +2, ...)
```

```
// This ensures IDs remain stable for the majority partition,
```

```
// which is important for downstream caching of community_score lookups.
```

```
RETURN [component for community in assignment.values()]
```

for component in connected_components(G.subgraph(community))]

Appendix B: CSA Weight Formula ? Parameter Sensitivity

The default parameter values ($\alpha=0.4$, $\beta=0.4$, $\gamma=0.1$, $\delta=0.05$, $\epsilon=0.05$) were

chosen based on the following intuitions:

Embedding similarity (α) and community membership (β) are given equal weight

because both capture complementary aspects of relevance: similarity captures

semantic proximity while community captures structural proximity.

Edge type (γ) is given lower weight because it is most useful in domain-

specific settings with rich edge type vocabularies.

Distance penalty (δ) is kept small to allow multi-hop paths without excessive

pruning.

Hop decay (ϵ) is minimal to allow deep traversal when needed.

In practice, $\alpha + \beta$ dominate the attention weights for most graphs. The other

parameters serve as tie-breakers and domain-adaptation handles.

Appendix C: Relationship to Existing KG Embedding Methods

TransE [Bordes et al., 2013] represents relations as translations in embedding

space: $\text{emb}(h) + \text{emb}(r) \approx \text{emb}(t)$ for (h, r, t) triples. Parallax can use

TransE embeddings directly for the similarity term in CSA without modification.

RotatE [Sun et al., 2019] represents relations as rotations in complex space.

More expressive for symmetric, antisymmetric, and compositional relations.

Also directly usable in Parallax.

Neither TransE nor RotatE produces multi-hop reasoning paths on their own.

Parallax uses their embeddings as the semantic grounding layer while the

traversal logic and community structure provide the reasoning.

End of white paper. Version 0.1 ? March 2026.

This document is the founding specification for the Parallax project.